

Guía de Defensa — Práctica 1

Procesamiento de Imágenes, Audio y Vídeo (PIAV) | Autores: Pablo Martínez & Daniel Rodríguez

1. Cumplimiento de Especificaciones (Calificación: 10 / 10)

Requisito del Enunciado	Puntos	Cómo se ha resuelto en el código	Estado
1a. Carga y RGB del píxel Cargar imagen y mostrar el RGB bajo el cursor.	2.5 pts	Lectura con <code>open_image()</code> . Evento <code><Motion></code> en el Canvas que indexa <code>image[y, x]</code> y muestra RGB: (R, G, B) adaptado al zoom.	CUMPLIDO
1b. Primitivas básicas Líneas, rectángulos, círculos, color RGB y grosor.	2.5 pts	Funciones en <code>drawing_tools.py</code> . Color RGB con 3 Spinboxes (0-255) y Grosor (1-50). Previsualización fluida al arrastrar sin ensuciar la imagen.	CUMPLIDO
2.1. Figuras rellenas Seleccionable desde la interfaz.	1.25 pts	Checkbox ' <i>Rellenar figura</i> '. Pasa <code>thickness=cv2.FILLED (-1)</code> a OpenCV o usa <code>cv2.fillPoly</code> en polígonos.	CUMPLIDO
2.2. Otras primitivas Elipses, polilíneas y polígonos.	1.25 pts	<code>draw_ellipse</code> (centro medio y semiejes), <code>draw_polyline</code> (abierta) y <code>draw_polygon</code> (cerrada). Se finalizan con doble clic.	CUMPLIDO
2.3. Guardar dibujo como vídeo Ver cómo se realizó paso a paso.	1.25 pts	Botón ' <i>Grabar vídeo</i> ' en la barra superior. Captura un fotograma clave tras cada dibujo y exporta un .mp4 con <code>video_recorder.py</code> .	CUMPLIDO
2.4. Aportación propia Mejoras y funciones adicionales.	1.25 pts	Zoom y Escala interactiva (25%-300% y autoajuste para imágenes grandes) + Filtros OpenCV (Grisés, Blur, Canny, Invertir) + Deshacer (Ctrl+Z) .	CUMPLIDO

2. Organización del Código (Diseño Modular Sencillo)

El proyecto se ha dividido en 5 archivos pequeños y muy claros, cada uno con una única responsabilidad:

- **main.py (7 líneas):** Punto de entrada. Crea la ventana principal de Tkinter y lanza la aplicación.
- **app.py (~270 líneas):** Controlador principal, interfaz gráfica, gestión de escala/zoom (25%-300%) y eventos del ratón.
- **drawing_tools.py (~45 líneas):** Librería de dibujo matemático con OpenCV: líneas, rectángulos, círculos, elipses y polígonos.
- **utils.py (~25 líneas):** Aportación propia: 4 filtros clásicos de visión por computador con OpenCV (Grisés, Blur, Canny, Invertir).
- **video_recorder.py (~20 líneas):** Exportación a vídeo MP4 con `cv2.VideoWriter` repitiendo fotogramas para visualización fluida.

3. Explicación de los Módulos del Código

A. app.py — Interfaz de Usuario, Zoom y Control del Ratón

Es el corazón de la aplicación. Gestiona la visualización adaptable y el flujo del dibujo:

- **Escala y Zoom a gusto del usuario:** Permite ajustar la imagen entre el 25% y el 300% con un deslizador, pulsar 'Ajustar' para encajar imágenes grandes en la pantalla, o '100%' para tamaño real. La función `get_coords()` divide automáticamente entre `scale_factor` para que siempre se dibuje e inspeccione sobre la matriz original exacta.
- **Al pulsar (<KeyPress-1>):** Obtiene la coordenada real con `get_coords(event)` y guarda el punto de inicio.
- **Al arrastrar (<B1-Motion>):** Dibuja sobre una copia temporal en memoria (`preview = self.image.copy()`) y refresca la pantalla. El usuario previsualiza el resultado sin modificar la imagen original.
- **Al soltar (<ButtonRelease-1>):** Guarda copia en el historial (Undo), estampa la figura en `self.image` definitiva y añade fotograma al vídeo si se está grabando.

B. drawing_tools.py — Primitivas Geométricas con OpenCV

Contiene funciones puras de dibujo analítico con OpenCV sin acoplamiento a Tkinter:

- **Línea:** Llama a `cv2.line(img, p1, p2, color, thickness)` con antialiasing para trazo suave.
- **Rectángulo:** Normaliza esquinas con `min()` y `max()` para que el rectángulo funcione perfectamente aunque arrastres de abajo hacia arriba o de derecha a izquierda.
- **Círculo:** Fija el centro en `p1` y calcula el radio con la distancia euclídea exacta: $\text{radio} = \text{int}(\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2})$.
- **Elipse:** Calcula el centro en el punto medio $((x_1+x_2)//2, (y_1+y_2)//2)$ y los semiejes según el desplazamiento $(|x_2-x_1|//2, |y_2-y_1|//2)$.
- **Polilíneas y Polígonos:** Convierte la lista de clics a array de NumPy (`np.int32`). Usa `cv2.polylines` (con `isClosed=False` para polilínea y `True` para polígono) o `cv2.fillPoly` si se marcó relleno.

C. utils.py — Filtros de Procesamiento de Imagen (Aportación Propia)

Integra 4 operaciones clásicas del temario de PIAV mediante funciones estándar de OpenCV:

- **Escala de Grises:** `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` (en 3 canales BGR para seguir dibujando en color encima).
- **Desenfoque (Blur):** `cv2.GaussianBlur(img, (9, 9), 0)`, suavizado gaussiano para atenuar ruido.
- **Bordes (Canny):** `cv2.Canny(img, 100, 200)`, detección óptima de gradientes y contornos.
- **Invertir Colores:** `cv2.bitwise_not(img)`, negativo fotográfico rápido.

D. video_recorder.py — Grabación Secuencial MP4

Exporta la lista de fotogramas clave con `cv2.VideoWriter` (códec mp4v a 4 FPS), repitiendo cada estado para poder examinar la secuencia paso a paso con comodidad.

4. Preguntas Típicas del Profesor y Respuestas Clave

Respuestas directas y técnicas para responder con total seguridad durante la evaluación:

P1. ¿Por qué al abrir una imagen con OpenCV obtienes los colores en orden BGR y no RGB?

Respuesta: OpenCV fue desarrollado por Intel a finales de los 90, cuando los formatos de captura analógica y frame grabbers ordenaban los bytes en Blue-Green-Red (BGR). OpenCV lo mantuvo por rendimiento. Al pasar a Tkinter o Pillow convertimos con `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`, y al leer un píxel `image[y, x]` reordenamos (b, g, r) a (r, g, b).

P2. ¿Cómo funciona la escala/zoom y cómo afecta a las coordenadas del ratón?

Respuesta: Al escalar la imagen para verla entera en pantalla (por ejemplo al 50%), las coordenadas del Canvas no coinciden con las de la matriz original. En `get_coords()` dividimos la posición del Canvas entre el factor de escala: `x = int(canvas.canvasx(event.x) / scale_factor)`. De esta forma, tanto el inspector RGB como las herramientas de dibujo trabajan siempre con precisión sobre los píxeles reales de la imagen.

P3. ¿Cómo consigues previsualizar la figura mientras arrastras el ratón sin estropear el dibujo?

Respuesta: Aplicando *doble buffer volátil*: durante el arrastre (<B1-Motion>) nunca modificamos `self.image`. En su lugar, clonamos una copia temporal `preview = self.image.copy()`, dibujamos sobre ella y refrescamos la vista. La imagen original solo se altera al soltar el ratón en <ButtonRelease-1>.

P4. ¿Cómo solucionas que el usuario dibuje un rectángulo arrastrando hacia atrás?

Respuesta: En `draw_rectangle()` normalizamos las esquinas calculando `pt1 = (min(x1, x2), min(y1, y2))` y `pt2 = (max(x1, x2), max(y1, y2))`. Así garantizamos que OpenCV siempre reciba la esquina superior izquierda y la inferior derecha en cualquier cuadrante de arrastre.

P5. ¿Por qué el vídeo del proceso no se graba de forma continua a 60 FPS?

Respuesta: Porque el enunciado pide guardar '*cómo se realizó paso a paso*'. Una grabación continua a 60 FPS contendría tiempos muertos mientras el usuario piensa o mueve el ratón. Capturando un fotograma clave al confirmar cada figura conseguimos un vídeo limpio, ligero y centrado en la secuencia de dibujo.

P6. ¿En qué consiste vuestra aportación propia?

Respuesta: Hemos implementado el sistema de zoom/escala interactivo para poder trabajar con imágenes de cualquier resolución (con autoajuste a la ventana), 4 filtros clásicos de OpenCV (Grises, Desenfoque, Canny e Inversión) y un sistema de Deshacer (Undo / Ctrl+Z).

5. Guión de Defensa en Vivo (Demostración en 2 Minutos)

Sigue estos sencillos pasos frente al profesor. Puedes ir diciendo la frase sugerida mientras haces cada clic:

Pa so	Acción en pantalla	Qué decir al profesor (Speech rápido)
1	Pulsar ' Abrir ' para cargar una imagen. Probar el deslizador de Zoom / Escala y el botón ' Ajustar '.	<i>'Cargamos la imagen. Vemos que si la imagen es grande se ajusta a la ventana, y con el control de zoom podemos escalarla a gusto para trabajar en cualquier zona sin perder precisión de píxel.'</i>
2	Mover el ratón sobre la imagen.	<i>'Al mover el ratón por el lienzo, el panel lateral muestra en tiempo real las coordenadas (x, y) reales y el valor exacto RGB del píxel bajo el cursor (Requisito 1a).'</i>
3	Pulsar ' Grabar vídeo '. Elegir color Azul y grosor 3. Dibujar Línea, Rectángulo y Círculo.	<i>'Iniciamos la grabación de vídeo. Seleccionamos color y grosor. Vemos que al arrastrar se previsualiza la figura sin manchar la imagen gracias al buffer temporal. Al soltar se confirma (Requisito 1b).'</i>
4	Marcar ' Rellenar figura '. Dibujar una Elipse rellena.	<i>'Activamos la opción de figuras rellenas con cv2.FILLED y trazamos una elipse inscrita calculando centro y semiejes automáticamente (Requisitos 2.1 y 2.2).'</i>
5	Elegir ' Polígono '. Hacer 4 clics en el lienzo y doble clic para cerrar.	<i>'Para polígonos, cada clic añade un vértice y al hacer doble clic se cierra y rellena automáticamente mediante cv2.fillPoly (Requisito 2.2).'</i>
6	Pulsar ' Grabar vídeo ' para guardar el .mp4. Aplicar filtro ' Bordes (Canny) ' y pulsar ' Deshacer '.	<i>'Detenemos la grabación y exportamos el MP4 paso a paso (Requisito 2.3). Finalmente, mostramos los filtros de OpenCV y el sistema de Deshacer (Requisito 2.4).'</i>

¡Todo listo para la máxima nota!

El código es conciso (~340 líneas en total entre todos los ficheros), cumple con el 100% de los requisitos del enunciado, permite escalar y trabajar cómodamente con imágenes grandes, y es sumamente fácil de entender y defender.